

TP1 : Expérimentation Attaques Web

Exercice 1

Dans cet exercice, vous allez faire des tests sur votre serveur HTTP en local qui héberge des fichiers PHP. Une vulnérabilité LFI (*Local File Include*) permet d'accéder au contenu de fichiers. Nous allons voir la partie attaque (auditeur/trice) et défense (webmaster) pour bien comprendre les LFI.

Le comportement normal de l'application est d'afficher le fichier `example.html` via le paramètre `file` de la page `preview.php`.

- 1) Depuis votre Kali Linux, démarrer le serveur HTTP Apache :

```
sudo -s  
systemctl restart apache2
```

- 2) Récupérer l'archive `Fichiers_PHP.zip` depuis la page moodle du cours et décompresser les fichiers dans `/var/www/html/` :

```
sudo 7z x Fichiers_PHP.zip -o/var/www/html/
```

- 3) La racine du serveur Web est par défaut dans `/var/www/html/`. Rester en utilisateur root. Le fichier `/var/www/html/preview.php` contient les données suivantes :

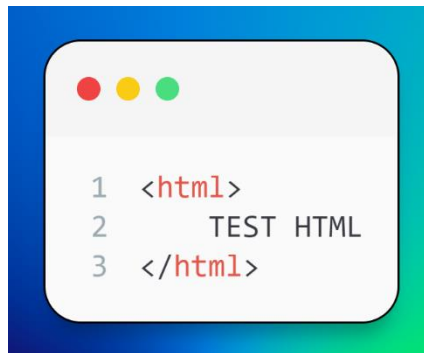


```
1 <?php  
2 ini_set('display_errors', 1);  
3  
4 include($_GET['file']);  
5 ?>
```

La ligne 2 permet d'afficher les erreurs sur la page. La ligne 4 affiche le fichier du paramètre GET « `file` ».

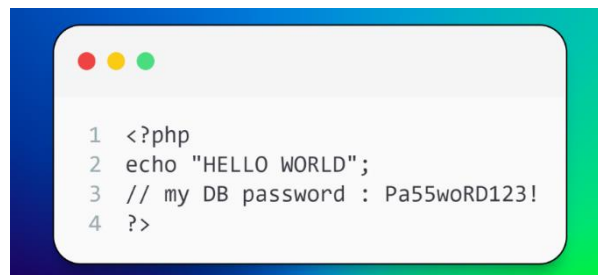
Vous allez exploiter la fonction `include()` de cette page.

4) Le fichier `/var/www/html/example.html` contient :

A code editor window with a blue header bar and three colored window control buttons (red, yellow, green) on the left. The editor contains three lines of HTML code:

```
1 <html>
2     TEST HTML
3 </html>
```

5) Le fichier `/var/www/html/hello.php` contient :

A code editor window with a blue header bar and three colored window control buttons (red, yellow, green) on the left. The editor contains four lines of PHP code:

```
1 <?php
2 echo "HELLO WORLD";
3 // my DB password : Pa55woRD123!
4 ?>
```

La ligne 2 affiche une chaîne de caractères (donc ici « HELLO WORLD ») sur le contenu HTML. La ligne 3 est un commentaire PHP.

6) Avec un navigateur Web, accéder à ce serveur HTTP :

<http://127.0.0.1/preview.php?file=example.html>

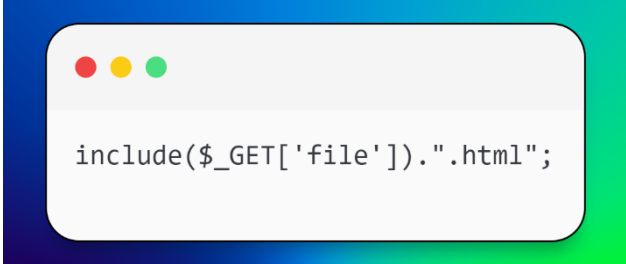
7) Provoquer une erreur : remplacer le nom de fichier par un nom inconnu, une erreur PHP doit s'afficher !

8) Essayer d'afficher le fichier `/etc/passwd` toujours en modifiant le paramètre `file`.

9) Essayer d'afficher le fichier `/etc/shadow`. Le fichier `preview.php` devrait avoir les droits de l'utilisateur « root » car vous l'avez créé à l'étape 2. Pourquoi n'est-il donc pas accessible ?

10) Toujours en modifiant le paramètre `file`, accéder au fichier `hello.php`. Pourquoi le fichier n'est-il pas entièrement affiché ?

- 11) Il est possible d'afficher le contenu d'un fichier PHP en utilisant la technique des *wrappers* PHP et de l'encodage base64. Rechercher des *payloads* possibles sur : <https://book.hacktricks.xyz/pentesting-web/file-inclusion> pour afficher la source du fichier `hello.php` via cette technique.
- 12) Il est parfois possible d'exécuter du code arbitraire grâce à une LFI. Aller voir le blog de Synacktiv (se concentrer sur le résultat pour exécuter `phpinfo()`) : <https://www.synacktiv.com/en/publications/php-filters-chain-what-is-it-and-how-to-use-it.html>
- 13) Utiliser cette payload pour afficher le résultat de la fonction `phpinfo()` avec le paramètre `file`.
- 14) Aller sur https://github.com/synacktiv/php_filter_chain_generator puis récupérer et exécuter le fichier `php_filter_chain_generator.py` au terminal avec les bons paramètres pour faire appel à `system($_GET['cmd'])` au lieu de `phpinfo()` puis utiliser le navigateur web pour exécuter les commandes suivantes : `pwd ; id ; ls`.
- 15) Nous allons modifier le fichier `preview.php` côté serveur pour s'assurer de ne récupérer que des fichiers HTML. Remplacer la ligne 6 par :



```
include($_GET['file']).".html";
```

En PHP, le caractère « . » permet de faire une concaténation. Donc ici, l'extension « .html » est ajoutée à la fin du nom du fichier demandé par le paramètre `file`.

- 16) Modifier le paramètre `file` pour accéder à `/etc/passwd`. Que se passe-t-il ? Est-ce que cette protection semble suffisante selon vous pour bloquer les tentatives d'accès aux autres fichiers (tels que `/etc/passwd` par exemple) ?
- 17) Rechercher des *payloads* possibles sur : <https://book.hacktricks.xyz/pentesting-web/file-inclusion> pour afficher la source du fichier `hello.php`

- 18) Si on remplace la ligne 6 de `preview.php` par celle-ci pour n'afficher que des fichiers du répertoire courant, comment peut-on contourner cette limite



Exercice 2

- 1) Récupérer le fichier suivant dans votre Kali :

<https://github.com/prasathmani/tinyfilemanager/archive/refs/tags/2.4.3.zip>

- 2) Décompressez-le dans `/var/www/html/tiny/` avec les droits 755
- 3) Créer un répertoire `/var/www/html/tiny/upload_files/` avec les droits 777 pour donner l'autorisation d'écriture pour tous
- 4) Le serveur doit être démarré (`service apache2 start`)
- 5) Ensuite, aller sur : `http://127.0.0.1/tiny/`
- 6) Télécharger l'exploit suivant puis exécutez le :

```
$ wget https://raw.githubusercontent.com/BKreisel/CVE-2021-45010/main/src/cve\_2021\_45010/main.py
```